LoRA4VR — Training Log

A living record of experiments, findings, and open questions

What this is. A dated log of experiments run on the LoRA4VR self-play loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result.

System. `Qwen2.5-Coder-3B-Instruct` fine-tuned with GRPO/DAPO on a Google Cloud L4 GPU (24 GB). The loop generates `policy(state) -> action` Python code, scores it by running the policy in a sandboxed environment, and trains LoRA adapters on the reward signal.

2026-04-13 Environment design and first training runs

`d96f7cb` · *puck-on-tilted-plane env, runs 1–3*

Motivation

CartPole was the original seed task but it is trivially solved by a two-line bang-bang policy, giving a flat learning curve from iteration 0. Any standard RL benchmark (LunarLander, Pendulum, MountainCar) is heavily represented in the base model's pretraining data, so improvement from LoRA would be confounded with prior knowledge. We need an environment the model has to *read* rather than recall.

The puck environment

A puck moves in a bounded $[-6, 6]^2$ arena under a constant hidden tilt (t_x, t_y) and linear drag μ. The policy receives $\{p_x, p_y, v_x, v_y, g_x, g_y\}$ and must drive the puck to the goal within 120 steps. Hidden parameters — tilt, drag, thrust limit $a_{\max}$, initial velocity — are sampled per scenario from a seed and are not in the state dict. The tight thrust envelope ($a_{\max} = \|tilt\|_2 \times 1.05 + \epsilon$) ensures that a naive "thrust straight at the goal" policy fails because it cannot cancel tilt drift.

Reward: $-\Delta t \cdot \text{dist}(p, g)$ per step; $+50$ bonus on reaching the goal. Errored episodes score -200 per scenario (floor).

Baseline gap (200 seeds, hand-written policies):

- Naive aim: $\approx 26\%$ goal rate
- PD + tilt feedforward: $\approx 73\%$ goal rate

This gap is real headroom for LoRA to exploit.

Bugs encountered and fixed

Three bugs had to be resolved before any learning was possible.

Run 1 — OOM on L4. Default config ($G = 16$, batch 16, max completion 900 tokens) ran out of memory at the backward pass. Fix (PR #6): halved G and batch to 8, trimmed completion length to 600.

Run 2 — Reward sign flip. Training ran but degraded (fewer running samples each iteration). Errored samples scored 0.0 while samples that ran and drifted integrated large negative rewards (≈ -280). GRPO's normalized advantage placed the 0.0 errors *above* the mean, so the trainer rewarded broken code. Fix (PR #7): errored scenarios score -200, making broken code strictly the worst outcome.

Run 3 — Dead advantages. With LR $= 10^{-5}$ and only 1 training prompt (4 gradient steps per outer iteration), $\approx 75\%$ of batches were uniformly errored (advantage $= 0$, no gradient). On the remaining steps, weight deltas were $\sim 10^{-6}$ — indistinguishable from zero. The reward was identical to 7 decimal places at iteration 0 and iteration 9. Fix (PR #10): LR bumped $10\times$ to 10^{-4}; epochs raised to 20 (20 gradient steps per outer iteration on 1 prompt).

Reproduce

Commits: OOM fix `ef228fe`, reward floor `ef7e0c9`, LR/epoch fix `e9353d7`. Env source: `scripts/seed_tasks/puck_tilted_plane_env.py`.

2026-04-18 Run 5 probe: base model vs. 5 iterations of GRPO

`d5bc81c` · *puck_grpo_run_5, iters 0 and 5, $n = 20$ samples each*

Setup

Run 5 used the config at `e9353d7` (LR $= 10^{-4}$, $G = 8$, 20 epochs per outer iteration, DAPO loss, $\beta = 0$). It completed 5 outer iterations before the VM was stopped. Adapter checkpoints were recovered to `models/lora/puck_grpo_run_5/`.

To inspect what the model actually writes, we sample 20 policies from the base model (iter 0, no adapters) and from the trained model (iter 5), score each against 8 scenarios, and compare. The tool is `scripts/probe_generation.py`; it saves per-sample policy files, a summary CSV, and per-scenario reward breakdown to `data/probe/{run_name}_iter{N}_t{temp}/`.

Results

	Ran OK	Crashed	Mean reward (runners)	Best reward
Iter 0 (base)	13 / 20	7 / 20	−218.9	−92.4
Iter 5	20 / 20	0 / 20	−140.4	−140.4

Iter 0

The base model generates varied PD/PID controllers but crashes 35% of the time by attempting to read hidden env internals (`a_max`, `dt`, `steps`) from the state dict. The 13 successful policies span a reward range of $[-92, -373]$ with genuine variation in gain choices. The best policy uses asymmetric gains ($k_{px} = 2.0, k_{dx} = 1.0$; $k_{py} = 1.5, k_{dy} = 1.0$) and scores -92.4 over 8 scenarios.

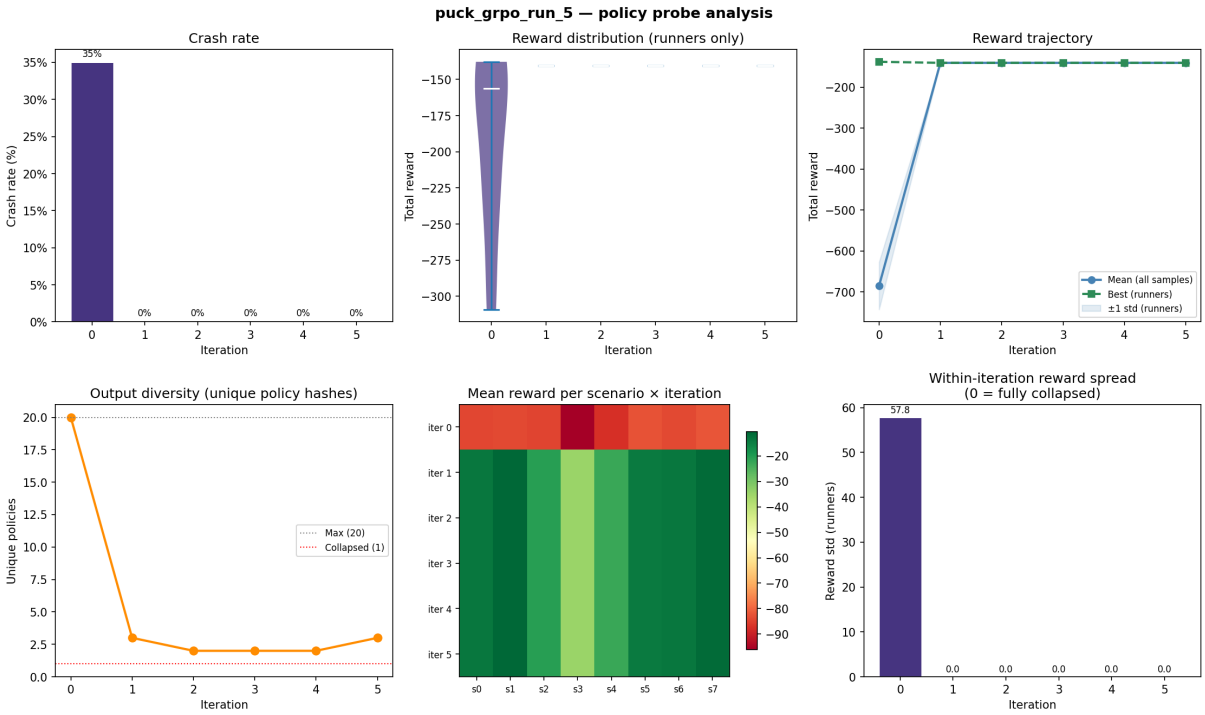


Figure 1: Run 5 probe analysis: base model (iter 0) vs. 5 iterations of GRPO. Generated by `scripts/analyze_probe.py --run_name puck_grpo_run_5`.

Iter 5

Zero crashes. The error-reading habit is gone. However, all 20 samples at temperature 0.8 produce *exactly* −140.3516 (identical to 4 decimal places). Every policy is byte-for-byte the same: $k_p = 2.0$, $k_d = 1.5$. This is the **illustrative example embedded in the prompt template** (`configs/prompt_template.txt`). The model has memorized and reproduced it verbatim.

Interpretation

Training achieved two things and failed at a third.

Achieved. Error rate dropped from 35% to 0% — the model learned not to read hidden env attributes. Mean reward over runners improved by 35%.

Failed. The model collapsed to a single policy. With $\beta = 0$ (no KL penalty), one training prompt, and 20 gradient steps per outer iteration, GRPO found the prompt example as a high-reward fixed point and reinforced it into a delta distribution. By iter 5, temperature 0.8 sampling produces zero diversity. The best reachable policy is now −140.4, worse than the base model's best (−92.4).

This is reward hacking: the model optimized the training distribution (memorize the example) rather than the environment.

Hypotheses to test

1. **Remove or paraphrase the prompt example.** It is the attractor GRPO is collapsing to. A deliberately suboptimal or absent example removes the fixed point.
2. **More training prompts.** With 1 prompt, every outer iteration trains on the same input. Adding 4–8 tasks with varied dynamics forces generalization.

3. **Raise $\beta > 0$.** A non-zero KL coefficient prevents the policy from drifting too far from the base distribution and preserves diversity.
4. **Reduce epochs.** 20 gradient steps on 1 prompt per outer iteration is aggressive. 5–10 may preserve more exploration.
5. **Probe at every intermediate checkpoint.** Does collapse happen at iter 1 or gradually? One probe per checkpoint would pinpoint when diversity dies.

Open questions

- What does the W&B reward curve look like across the 5 iterations? (The run at `puck_grpo_run_5-orchestrator` has data but some post-resume steps may be missing due to a step-monotonicity issue.)
- What is the PD+feedforward ceiling expressed in probe-script reward units? This would establish what score a successful LoRA policy needs to beat.

Reproduce

At commit `d5bc81c`, with adapter checkpoints in `models/lora/puck_grpo_run_5/`:

```
scripts/probe_generation.py --iter 0 --run_name puck_grpo_run_5 --n 20
```

```
scripts/probe_generation.py --iter 5 --run_name puck_grpo_run_5 --n 20
```

Results land in `data/probe/puck_grpo_run_5_iter{000,005}_t0p80/`.

```
scripts/analyze_probe.py --run_name puck_grpo_run_5
```

2026-04-19 Run 8: removing the prompt example unlocks genuine learning

`7ce5526` · *puck_grpo_run_8, 28 outer iterations, iters 0/4/8/12/16/20/24/27 probed ($n = 20$ each)*

Setup

Run 7 (aborted at iter 1) confirmed the model was already generating diverse policies without the prompt example. Run 8 ran the full 30-iteration schedule on the same config as run 5 (`e9353d7`), with the single change that the illustrative $k_p = 2.0$, $k_d = 1.5$ example was removed from `configs/prompt_template.txt` (PR #13, `e5fe162`). Adapters were downloaded from the GCP VM and placed in `models/lora/puck_grpo_run_8/`. Eight evenly-spaced iterations were probed to capture the full learning trajectory without evaluating all 28.

Results

Interpretation

This is the first run showing genuine learning. Three phases are visible. *Iters 0–4*: crashes drop from 85% to 0% as the model learns the valid state-key constraint. *Iters 4–16*: reward improves steadily from −221 to near zero — the model is reaching the goal on some scenarios but not consistently. *Iters 16–20*: a sharp transition to positive reward (+199 mean), indicating consistent goal reaching across most scenarios. The curve flattens after iter 20 with marginal gains through iter 27.

The single change responsible is the removal of the prompt example. In run 5 the example provided a ready-made fixed point that GRPO collapsed to after one update. Without it, the model must explore the policy space, and GRPO has real variance to amplify.

Iter	Crashes	Mean reward	Best reward
0	17 / 20	−1403	−223
4	0 / 20	−221	−165
8	0 / 20	−69	−64
12	0 / 20	−67	−54
16	0 / 20	−28	+12
20	0 / 20	+199	+248
24	0 / 20	+212	+247
27	0 / 20	+215	+248

Table 1: 20 samples × 8 scenarios each. Reward > 0 indicates consistent goal reaching (goal bonus = +50 per scenario).

The iter-27 best reward of +248 approaches the hand-written PD+feedforward ceiling (estimated at ≈ +270 for 8 scenarios at +50 goal bonus each, assuming ∼ 73% goal rate from the 200-seed baseline).

Hypotheses / next steps

1. **Probe all 28 iterations** to pinpoint exactly when the phase transition at iter 16–20 occurs and whether it is sharp or gradual.
2. **Measure diversity at convergence.** Run 5 collapsed to 1 unique policy; does run 8 maintain diversity at iter 27, or does it collapse to a different fixed point once reward saturates?
3. **Compare against the hand-written ceiling.** Score the PD+feedforward baseline with `scripts/probe_generation.py` at iter 0 to get a reward number in the same units.
4. **Train longer or with more prompts** to see whether the plateau at iter 20–27 is a capacity limit or simply convergence of the single training task.

Reproduce

At `7ce5526`, with adapters in `models/lora/puck_grpo_run_8/`:

```
scripts/probe_all_iters.py --run_name puck_grpo_run_8 --iters 0 4 8 12 16 20 24 27
--n 20

scripts/analyze_probe.py --run_name puck_grpo_run_8
```

2026-04-22 Policy analysis: what run 8 learned, and why iter 0 failed

`e6bc200` · *manual probe inspection, run 8 iter 0 and iter 27*

Setup

With run 8 showing genuine learning (mean reward +215 at iter 27, up from −1403 at iter 0), the question was: what strategy did the model actually discover, and why was the base model unable to write a working policy at all? All 20 iter-0 and all 20 iter-27 policy files from `data/probe/puck_grpo_run_8_iter{000,027}_t0p80/` were read and compared manually.

Results

Iter 27 — convergent velocity P-controller. Every iter-27 sample uses the same structural template: compute the angle to the goal, set a desired velocity of magnitude *speed* in that

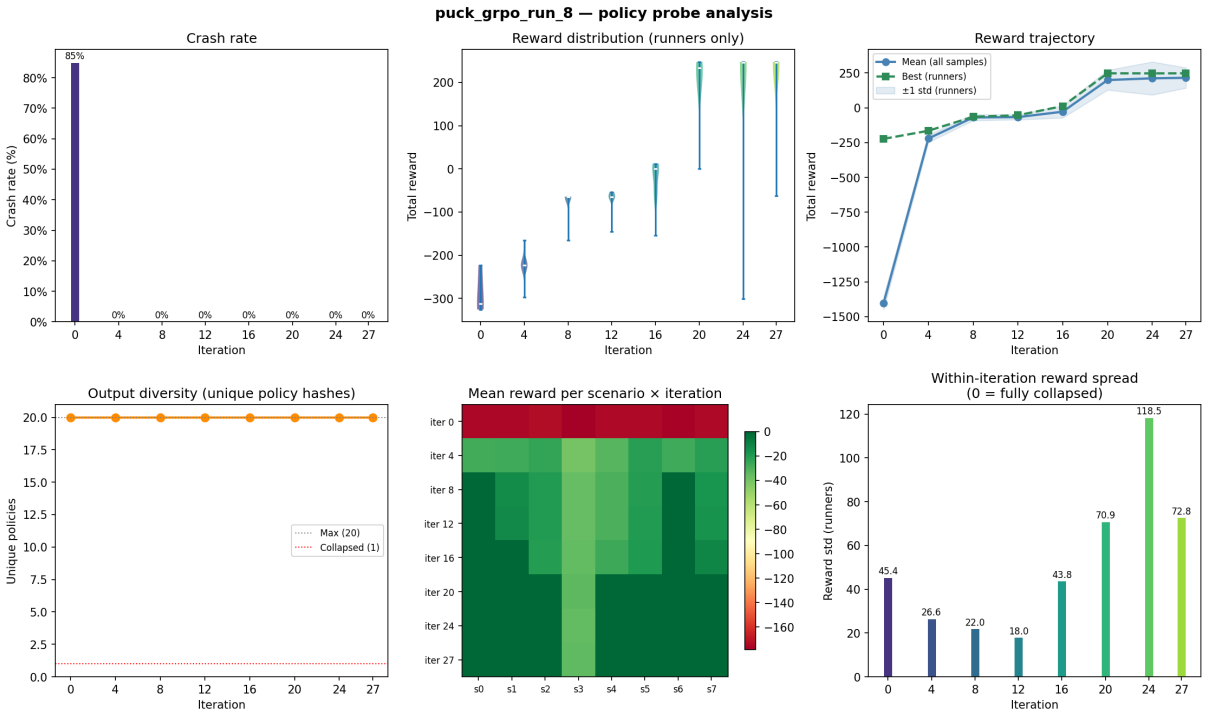


Figure 2: Run 8 learning curve: mean and best reward across 8 probed iterations ($n = 20$ samples each). Generated by `scripts/analyze_probe.py --run_name puck_grpo_run_8`.

direction, and command acceleration proportional to the velocity error:

$$a = \text{gain} \times (v_{\text{desired}} - v_{\text{current}})$$

The only variation across the 20 samples is in two scalar hyperparameters: desired speed (1.0–5.0) and gain ($/0.05 = \times 20$, $\times 10$, $\times 20$). The three below-average samples each have a distinct defect: sample 08 contains a positional-step bug that degenerates to naive aim (reward -62.8); sample 10 sets speed $= 1.0$, too slow to reach the goal in 120 steps; sample 15 self-clips at 3.0 below the env's own $a_{\max}$.

Iter 0 — API mismatch causes 85% crash rate. 17 of 20 base-model policies crashed with KeyError at reward floor -1600. The specific hidden keys accessed are summarised in Table 2. The three survivors avoided hidden keys only by accident: two hardcoded thrust magnitude constants and one hardcoded all physics parameters (`tilt_x = 0.0`, `mu = 0.5`, `a_max = 1.5`). The survivors scored -223, -312, and -325 — all worse than the naive aim baseline.

Hidden key	Samples accessing it
`a_max`	00, 01, 02, 03, 04, 05, 07, 08, 11, 12, 14, 15, 16, 19
`tilt_x`, `tilt_y`	01, 03, 04, 06, 08, 11, 12, 13, 14, 15, 16, 19
`mu`	01, 04, 07, 08, 12, 15, 16
`dt`	16, 17

Table 2: Hidden state keys accessed by crashing iter-0 samples.

Interpretation

What iter 27 discovered. The trained model found a velocity P-controller rather than the PD-on-position + tilt-feedforward strategy used by the hand-written baseline. This is structurally

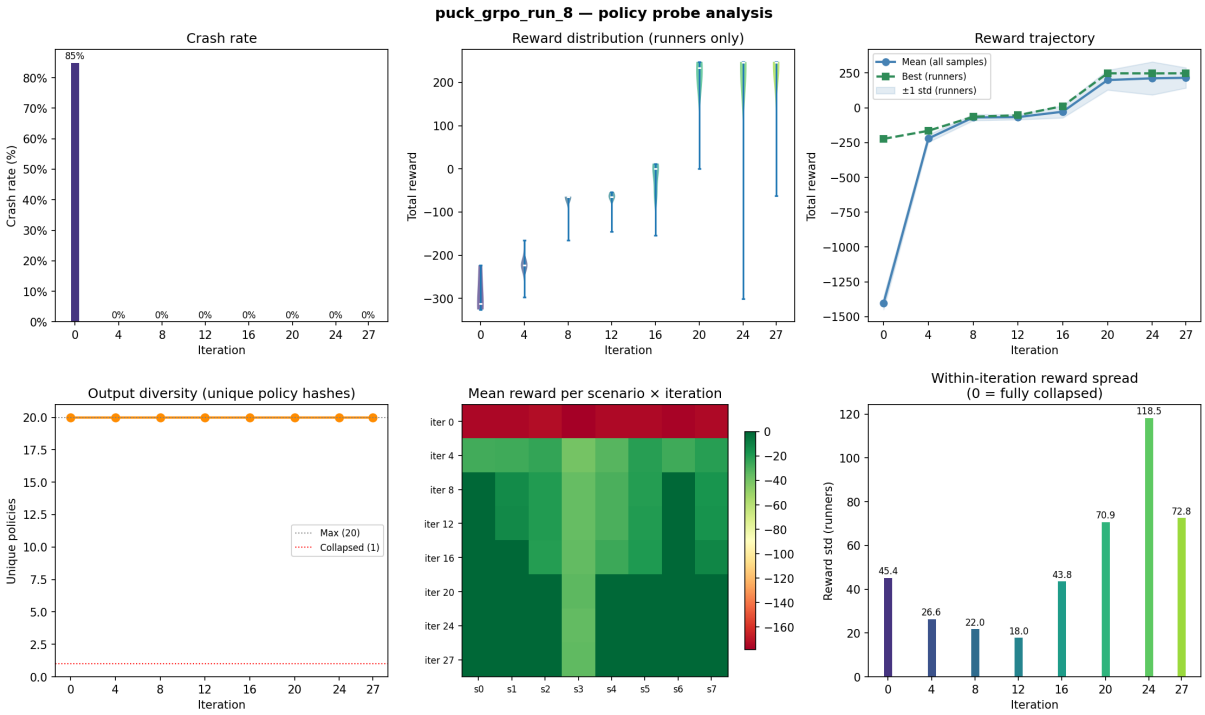


Figure 3: Run 8 learning curve across the 8 probed iterations, reproduced here for context. The phase transition at iter 16–20 is the backdrop for the policy structure analysis above.

different: there is no position error term and no tilt awareness. The approach works because high-gain velocity tracking rejects tilt disturbances reactively within a few timesteps, yielding near-ceiling reward without explicit feedforward. The residual 8% gap to the hand-written ceiling (+248 vs. ≈ +270) is consistent with reactive rather than proactive tilt rejection.

Why iter 0 failed. The base model's prior — drawn from pretraining on standard RL codebases — is that a physics simulation exposes all its parameters through the state dict or environment attributes. 14 of 17 crashing samples attempted tilt compensation or drag cancellation that would have been correct if the hidden parameters were accessible. The model's control-theory knowledge was largely sound; its environment-API assumption was wrong. GRPO fixed the API assumption without teaching better control theory, because velocity tracking was sufficient to saturate the available reward.

Hypotheses / next steps

1. **Probe the transition from position-thrust (iter 0 survivors) to velocity-tracking (iter 27).** At which iteration does the velocity-control template first appear? Checking iter-4 policies would show whether it emerges immediately after crash rate drops.
2. **Test whether harder tilt distributions force tilt compensation.** With wider tilt ranges the reactive velocity-tracker may fail; GRPO would then need to discover an explicit feedforward term to push reward higher.
3. **Provide the hidden parameters in the state dict and retrain.** If the base model already knows how to use tilt/drag/a_max (most iter-0 samples attempt it), a single run with those keys exposed could achieve the hand-written ceiling immediately and reveal what further headroom exists beyond it.

Open questions

- Would the velocity-tracking template generalise to a new environment with different dynamics, or is it a memorised pattern for the puck task?
- The best iter-27 policy (+247.6) and the best iter-0 survivor (−223) both use only valid state keys. Is the iter-27 template a refinement of the iter-0 survivor template, or did it emerge independently under GRPO pressure?

Reproduce

Manual inspection at `e6bc200`; policy files already present in `data/probe/` (run 8, iter 0 and iter 27 probe directories).

2026-04-22 Phase transition iter 16→20: gain and speed jointly determine goal capture

`cd60244` · *manual inspection of iter016 and iter020 probe files, run 8*

Setup

The run-8 learning curve shows a sharp mean-reward jump from +28 (iter 16) to +199 (iter 20). To understand the cause, all 40 policy files from the run-8 iter-16 and iter-20 probe directories were read and compared.

Results

Both iterations use the velocity-tracking template without exception: no crashes, no hidden-key reads. The difference lies entirely in the choice of two scalar hyperparameters.

Iter	Dominant speed	Dominant gain	Mean reward	Best reward
16	10.0	×1	+28	+12
20	2.0–5.0	×10–50	+199	+248

At iter 16, 14 of 20 samples set `desired_speed = 10` and compute acceleration as the raw velocity error (`ax = desired_vx - vx`, gain = 1). Five samples compute the desired velocity as an unnormalized position vector (`desired_vx = speed * dx`), giving a distance-proportional target that oscillates near the goal; these score −53 to −154. By iter 20, all samples use speeds of 1.5–5.0 and gains of ×10–×50 (via `/ 0.05`, `* 10.0`, `* 20.0`, or chained multipliers). The one failure at iter 20 (sample 14, reward +0.5) is the sole sample that reverts to `desired_speed = 10` with gain ×10.

Interpretation

The controller has no explicit distance term and does not decelerate near the goal. Goal capture depends on two mechanisms.

Finite goal-zone radius. The puck need only be inside the goal radius at one timestep. At speed v and $dt = 0.05\,\text{s}$, the per-step displacement is $v \cdot dt$. At 3 m/s this is 0.15 m, comfortably below the goal diameter ($\approx 0.5\,\text{m}$), guaranteeing detection on a direct pass.

Tilt rejection and overshoot amplitude. After the puck passes the goal, `angle_to_goal` flips immediately; the controller commands a reversal. The overshoot amplitude before the reversal takes effect scales as $v_{\text{desired}}/\text{gain}$: low speed and high gain together keep the oscillation amplitude small so the puck re-enters the goal zone on the next pass. More importantly, during the approach,

a lateral tilt acceleration $\mathbf{g}_\perp$ induces a sideways velocity error δv. With gain = 1, the correction is δv (which clips immediately at $a_{\max}$, providing no extra correction). With gain = 20, the correction is $20\,\delta v$ in the linear regime, rejecting tilt drift 20× faster and keeping the trajectory aligned with the goal despite hidden tilt.

The iter-16 failure is therefore not discrete-timestep miss (step size 0.2 m at terminal velocity ≈ 4 m/s is still inside the goal diameter) but poor tilt rejection: with gain = 1 the puck drifts off the goal axis and misses the zone on most scenarios. GRPO resolved this by selectively reinforcing samples where the gain happened to be high enough to overcome tilt drift, receiving the sharp +50 goal bonus as the reward signal.

Hypotheses / next steps

1. **Probe iter 17–19** to determine whether the gain shift happens in one step or gradually. The +50 bonus is a discontinuous signal that could cause a single-iteration jump if the right (speed, gain) combination appears at random and is then strongly reinforced.
2. **Measure whether gain or speed is the primary driver.** Evaluate policies with high gain + high speed, and low gain + low speed, on the 8-scenario test set to separate the tilt-rejection and step-size effects.
3. **Test with wider tilt distributions.** If tilt rejection is the bottleneck, harder tilt scenarios should widen the gap between gain = 1 and gain = 20 controllers, making the transition more pronounced.

Open questions

- The transition occurs at a specific GRPO iteration. Is there a single lucky sample with high gain that crossed the reward threshold and was amplified, or did gain drift upward continuously across iter 16–20?
- What is the minimum gain needed for reliable goal capture at a given tilt magnitude? This would give a quantitative criterion for when the velocity-tracking controller is "good enough."

Reproduce

Manual inspection at `cd60244`; policy files already present in `data/probe/` (run 8, iter 16 and iter 20 probe directories).

2026-04-27 Branched-rollout N=2 OOMs on L4; plan to decompose loss per turn

`8b94cc7` · *branched-N=2 GRPO smoke, PR #16*

Setup / motivation

After run 8 produced a clean single-turn baseline, the next experiment is N=2 multi-turn GRPO with branched MC rollouts (G_1 first-turn variants, G_2 continuations each), using turn-2 mean reward as an empirical $\hat{V}_1$ to give turn-1 a credit-assignment signal without a learned critic. The override (`src/branched_trainer.py`) plugs into TRL 0.28's `GRPOTrainer` by replacing `_generate_and_score_completions` and concatenating each trajectory as [prompt | turn1 | tool | turn2] before handing it to the parent's `_compute_loss`. The smoke goal was an end-to-end GPU run on L4 (24 GB) to validate plumbing before scaling.

Results

Three smoke attempts on L4, all OOMed at the same point: a 4.06 GiB allocation inside the autograd backward pass, ≈ 3 min 30 s into the run. The allocation size was identical across attempts despite progressive shrinking of the config:

Attempt	G_1	G_2	max_turn_tokens	Outcome
1 (50825d3)	4	2	400	OOM at backward, 4.06 GiB
2 (59f4d81)	2	2	200	OOM at backward, 4.06 GiB
3 (f735d1d)	2	2	100	OOM at backward, 4.06 GiB
4 (8b94cc7)	2	2	200	OOM at backward, 4.06 GiB (prompt trimmed)

Trimming the Multi-Turn Protocol prose from the prompt freed ≈ 300 MiB at peak (free memory 2.98 → 3.29 GiB) but did not move the 4.06 GiB allocation. Generation, sandbox execution, advantage computation, tool/loss-mask assembly, prompt-completion concatenation, and logp computation all completed — the override reaches `loss.backward()` on every attempt before failing.

Interpretation

The plumbing is sound; the structural issue is the loss layout. The 4.06 GiB allocation is the logits gradient tensor at backward, shape $(B, T_{\text{completion}}, V)$ with $V = 151{,}936$. The branched-N=2 setup makes both factors larger than single-turn:

- Per-row sequence is now [prompt | turn1 | tool | turn2], with the prompt itself enlarged by the multi-turn protocol description, scenarios, and env code (single-turn prompts were ≈ 500 tokens; branched prompts approach 1500–2000).
- The completion now spans turn-1 + tool result + turn-2 rather than a single turn.
- Backward retains activations across the full sequence at every transformer layer (36 layers, $H = 2048$); attention activations grow super-linearly in T.
- All $G_1 G_2$ trajectories run in one forward+backward graph, so memory peaks as a sum, not a max.

The single-turn G=16 baseline fit because per-row $T \approx 900$ and the prompt was short. Branched G=4 does not fit because per-row $T \approx 2500$ and the multi-turn prompt is large.

Hypotheses / next steps

The next task is *Option C: per-turn loss decomposition*, recorded in detail in the tracker task `nontrivial-envs/per-turn-loss-decomposition`.

1. **Split the forward into two passes.** Turn-1 logps need only [prompt | turn1] (G_1 short forwards, no G_2 redundancy). Turn-2 logps still need [prompt | turn1 | tool | turn2] but use `logits_to_keep` so the LM head only materializes (B, T_{turn2}, V) rather than the full completion span.
2. **Backward each pass separately.** Two sequential `.backward()` calls with `retain_graph=False` accumulate gradients into the same parameters; peak memory becomes max of the two passes rather than their sum.
3. **Custom `_compute_loss`.** Re-implement the DAPO loss segment-by-segment (≈ 30 lines) since TRL's parent expects one batched advantage tensor. Use the additive decomposition $\mathcal{L}_{ij} = \mathcal{L}_{ij}^{(1)} + \mathcal{L}_{ij}^{(2)}$ with the G_2 scaling on turn-1 to match the original sum.
4. **Optionally micro-batch turn-2** below $G_1 G_2$ to bound peak further if production sizing ($G_1 = G_2 = 4$, $T = 400$) still OOMs after the split.

Memory analysis (back-of-envelope, dominant logits-grad term only, at production sizing $G_1 = G_2 = 4$, $T_1 = T_2 = 400$, $T_t = 100$): current implementation ≈ 4.4 GiB; Option C peak $\approx \max(0.49, 1.94) = 1.94$ GiB. With turn-2 micro-batching at $b = 4$, peak drops to ≈ 0.49 GiB.

Open questions

- Will the segmented forward correctly reproduce the concatenated loss numerically? A CPU unit test on a fixed input is in the task acceptance criteria; the math decomposes cleanly because tool tokens have $m_t = 0$, but tokenization-seam offsets between [prompt | turn1] and the full chat could shift boundaries and need careful handling via `apply_chat_template` on the full message list with index tracking.
- Does the additional forward (turn-1-only pass) cost more wall-time than it saves via the G_2-fold turn-1 redundancy reduction? Expected to be roughly neutral; confirm empirically.
- Generalization to $N \geq 3$: each turn gets its own forward+backward at $\prod_{k<n} G_k$ unique sequences. The pattern composes but compute grows multiplicatively per added turn.

Reproduce

Smoke config at `configs/smoke_test_grpo_branched.yaml`; run via `python scripts/09_run_loop.py --config configs/smoke_test_grpo_branched.yaml`. The OOM is reproducible at `8b94cc7` on a single L4 (24 GB).

2026-04-29 Two-pass per-turn loss decomposition fits on L4

`9a1c296` · *Option C lands; branched-N=2 smoke completes end-to-end*

Setup

Implemented Option C from the prior entry: split the single forward over [prompt | turn1 | tool | turn2] into two sequential passes. Pass 1 forwards [prompt | turn1] and backwards the turn-1 DAPO loss; pass 2 forwards the full prefix [prompt | turn1 | tool] as the "prompt" with turn-2 alone as the "completion", so the parent's `logits_to_keep` becomes T_{turn2} rather than $T_{\text{completion}}$. Both passes share the same `num_items_in_batch` normalizer, making $\mathcal{L} = \mathcal{L}^{(1)} + \mathcal{L}^{(2)}$ exact for additive loss types. Backward runs inside the overridden `compute_loss`, with `training_step` skipping the outer `accelerator.backward(loss)`. Restricted to `loss_type` $\in$ {`dapo`, `cispo`}, token-level IS, $\beta = 0$, `num_iterations` $= 1$; per-row averaging losses don't decompose this way without a different normalizer scheme.

Results

Smoke config (`configs/smoke_test_grpo_branched.yaml`, $G_1 = G_2 = 2$, `max_turn_tokens` $= 200$) ran 4 training steps end-to-end on an L4 in `europe-west3-b` with exit code 0. Same config OOMs at backward on `8b94cc7` (parent of this branch) per the previous entry. The peak-memory delta was not instrumented this run — the binary fits-vs-OOMs signal at identical sizing was the test.

Interpretation

The dominant 4.06 GiB allocation in the prior OOM was the LM-head logits gradient at backward, shape $(B, T_{\text{completion}}, V)$. Splitting the backward graph at the turn boundary makes peak memory $\max(\text{pass}_1, \text{pass}_2)$ instead of their sum: pass 1's activations are released before pass 2's forward begins. Since tool tokens carry $m_t = 0$ in the loss mask, no gradient signal is lost by the split.

Hypotheses / next steps

1. Bump G_1, G_2 on the production config and re-measure — the memory headroom should now allow $G_1 G_2 \geq 8$ at $T \approx 200$ where single-pass previously OOMed.
2. Add `torch.cuda.max_memory_allocated()` logging to `training_step` to quantify the reduction (currently the win is inferred, not measured).
3. Lift the `num_iterations = 1` restriction by slicing `old_per_token_logps` per segment with the same right-aligned layout as the segment forwards.
4. Per-segment `ref_per_token_logps` plumbing to support $\beta \neq 0$ (KL term).

Open questions

- Wall-time delta vs. single-pass: two forwards instead of one, but the turn-1 forward is much shorter. Net unmeasured.
- Generalization to $N \geq 3$ turns: the additive decomposition composes, but each added turn adds another forward+backward whose prefix grows.

Reproduce

`9a1c296` on `feat/per-turn-loss-decomposition`; launch via `bash scripts/gcp_find_l4_and_train.sh configs/smoke_test_grpo_branched.yaml two_pass_smoke --wait`. Decomposition logic in `src/branched_trainer.py` (`compute_loss`, `training_step`, `build_segment_tensors`); unit tests in `tests/test_segment_tensors.py`.

2026-05-09 Slosh-tank env introduced; dimensionless analysis guides parameter selection

New environment, branch `main`; env source `scripts/seed_tasks/slosh_tank_env.py`

Decision: pivot from harder-puck to slosh tank

The planned harder-puck variant (small $\mu = 0.05$, wider tilt, tight headroom, two-state policy interface) was abandoned before any code was committed. The core problem: the signal of interest — tilt estimation from transition residuals — requires the model to compute $(v_{\text{new}} - v_{\text{prev}})/dt - a \approx$ tilt, a mathematical identity. The environment cannot test *how well* the model uses that identity because a policy that writes it verbatim from the prompt succeeds trivially. There is no gradient to climb.

The slosh-tank replaces it. The hidden unknowns affect *how* to control — deadband changes command style, delay changes timing, slosh length changes feedback gain tuning — and no single formula bridges the gap. The probe reveals the unknown; the model must then write a qualitatively adapted controller. This is a harder reasoning task with a clearer reward surface.

Environment summary

A 1D cart (x, v) carries liquid modelled as one damped pendulum mode (θ, ω). Policy commands horizontal acceleration, clipped to $[-1, 1]\,\text{m/s}^2$. Hidden per-scenario parameters: actuator gain κ, action delay D (steps), deadband b, slosh length L, damping c. Observed state: $\{t, x, v, \theta, \omega, g, \text{steps_left}\}$. Dynamics: cart motion plus a forced damped pendulum with coupling $\dot{\omega} = -(g/L)\theta - c\,\omega - a_{\text{true}}$. Success requires all four conditions: $|x - g| < 0.12$, $|v| < 0.12$, $|\theta| < 0.07$, $|\omega| < 0.18$. The four-condition criterion prevents degenerate solutions (reach position without settling liquid). Episode length: $N_{\text{steps}} = 105$, $\Delta t = 0.06\,\text{s}$ (6.3 s total).

Source: `scripts/seed_tasks/slosh_tank_env.py`. Difficulty ladder and curriculum: `slosh_tank_difficulty_`

Dimensionless analysis via Buckingham π

We applied the Buckingham π theorem directly to the physical parameters, setting aside the integration step, success thresholds, and RL reward weights (those belong to the measurement and training layers, not the dynamics).

The 8 governing parameters.

Parameter	Symbol	Value	Dimension
Gravitational accel.	g	$9.81\,\mathrm{m\,s^{-2}}$	$\mathsf{L}\,\mathsf{T}^{-2}$
Max commanded accel.	$a_{\max}$	$1.00\,\mathrm{m\,s^{-2}}$	$\mathsf{L}\,\mathsf{T}^{-2}$
Cart→slosh coupling	η	$1.00\,\mathrm{m^{-1}}$	L^{-1}
Slosh pendulum length	L	$0.70\,\mathrm{m}$	L
Slosh damping rate	c	$0.06\,\mathrm{s^{-1}}$	T^{-1}
Goal distance	D_g	$\sim 1.8\,\mathrm{m}$	L
Episode time	T_{ep}	$6.3\,\mathrm{s}$	T
Actuator gain (hidden)	κ	$\{0.65, 0.80, 1.00\}$	1

Buckingham π (repeating vars: $a_{\max}$, D_g). 7 dimensional params, 2 fundamental dims $\Rightarrow$ 5 new groups + κ:

$$\pi_1 = \frac{g}{a_{\max}} = 9.81, \quad \pi_2 = \eta D_g = 1.80, \quad \pi_3 = \frac{L}{D_g} = 0.39, \quad \pi_4 = c\sqrt{\frac{D_g}{a_{\max}}} = 0.080, \quad \pi_5 = T_{\mathrm{ep}}\sqrt{\frac{a_{\max}}{D_g}} = 4.70.$$

Meaningful recombinations.

$$T_{\mathrm{tr}} = 2\sqrt{D_g/a_{\max}} = 2.68\,\mathrm{s}, \qquad \tau = \tfrac{1}{2}\pi_5 = \frac{T_{\mathrm{ep}}}{T_{\mathrm{tr}}} = 2.35,$$

$$N_s = \frac{1}{\pi}\sqrt{\frac{\pi_1}{\pi_3}} = \frac{T_{\mathrm{tr}}}{T_s} = 1.60 \text{ (slosh cycles/transit)}, \qquad \zeta = \frac{\pi_4}{2\sqrt{\pi_1/\pi_3}} = \frac{c}{2\omega_n} = 0.008,$$

$$\theta_{\mathrm{ss}} = \frac{\pi_2\,\pi_3}{\pi_1} = \frac{\eta\, a_{\max}\, L}{g} = 0.071 \text{ rad}.$$

θ_{ss} is the equilibrium slosh angle under sustained full thrust. No tolerance parameter appears in its formula, yet $\theta_{\mathrm{ss}} \approx \epsilon_\theta = 0.07\,\mathrm{rad}$ — a collision between the dynamics and the success condition that falls directly out of the π groups. It means any final approach at full commanded acceleration violates the slosh constraint; near-goal corrections must be gentle.

Confirmed V1 configuration and empirical gap

V1: actuator gain only at `max_steps=105` ($\tau = 2.35$).

Parameter	Symbol	V1 value	Note
Episode steps	N_{steps}	105	$\tau = 2.35$
Actuator gain	κ	{0.65, 0.80, 1.00}	V1 unknown
Goal distance	D_g	$[1.4, 2.2]\,\mathrm{m}$	unchanged
Action delay	D	0 steps	fixed
Deadband	b	0	fixed
Slosh length	L	$0.70\,\mathrm{m}$	fixed
Damping	c	$0.06\,\mathrm{s^{-1}}$	fixed

Empirically confirmed gap (200 seeds per κ). Sweep of blind nominal PD vs. oracle (PD gains divided by κ) at `max_steps`=105, goal= $[1.4, 2.2]$ m, $L = 0.70$ m:

κ	Blind success	Oracle success
0.65	0%	100%
0.70	0%	100%
0.75	25.5%	100%
0.80	50.5%	100%
0.85	85%	100%
0.90	100%	100%

The gap is not a transit-timeout effect (condition (**??**) requires $\tau < 1.24$, but $\tau = 2.35$). It arises through settling: at $\kappa < 1$ the actuator delivers κ-attenuated corrections in the near-goal linear regime, slowing convergence in position, velocity, and slosh simultaneously. The oracle compensates by scaling its command by $1/\kappa$, restoring the intended correction magnitude.

This gap satisfies the minimum requirement: blind vs. oracle separation exceeds 20% across $\kappa \in \{0.65, 0.80\}$, with oracle at 100% for all $\kappa \geq 0.65$.

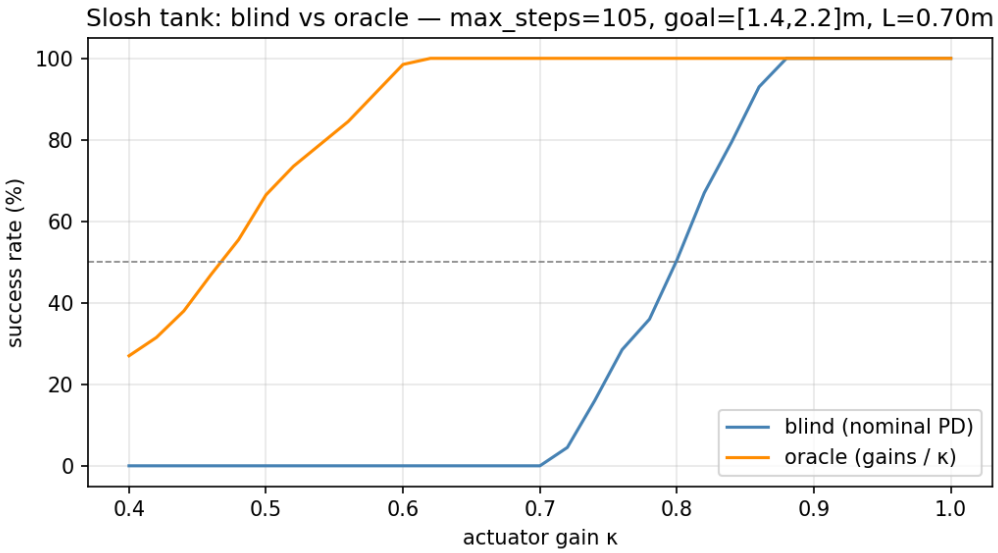


Figure 4: Blind (nominal PD) vs. oracle (gains/κ) success rate over 200 seeds at each κ level. `max_steps`=105, goal= $[1.4, 2.2]$ m, $L = 0.70$ m. Generated by `scripts/gain_sweep.py`.

Reproduce

`scripts/gain_sweep.py` — runs 200 seeds × 13 κ levels, saves `docs/slosh_gain_sweep.png`. Env source: `scripts/seed_tasks/slosh_tank_env.py` at current `main` (`max_steps`=105, $\kappa \in \{0.65, 0.80, 1.00\}$ in `sample_hidden`).

Open questions / next steps

1. **Confirm gap at $\tau \approx 1.4$.** Implement the recommended `max_steps` and unknown ranges; run blind, oracle, and probe-then-adapt baselines on 200 seeds.
2. **Design probe-then-adapt oracle.** The hand-written oracle currently adjusts gains only; it does not plan trajectories. For action delay, a proper oracle uses a bang-bang plan with delay-shifted braking point. This determines the achievable two-turn ceiling.

3. **First GRPO run.** Once the gap is confirmed: use same config as run 8 (LR = 10^{-4}, DAPO loss, $\beta = 0$) on V1 (action delay only). Probe at iters 0, 4, 8, 16, 20 to track whether the model learns to write a probe in turn 1 and adapt in turn 2.
4. **Does the model learn to probe at all?** The key empirical question: does the trained model's turn-1 function look like the canonical diagnostic pulse, or does it collapse to a fixed controller? The turn-1 reward curve (mean reward of turn-1 draft against training step) is the primary diagnostic.

2026-05-10 Arm A: frozen one-shot Qwen2.5-Coder-3B baseline

Branch `main`; scripts `scripts/eval_frozen_baseline.py`, `scripts/gcp_find_gpu_and_eval.sh`. Run: `arm_a_001`, 20 samples, 50 seeds, L4 GPU.

Setup

Base Qwen2.5-Coder-3B-Instruct (no LoRA), single-turn prompt (`total_turns=1`), temperature 0.8. Each of the 20 sampled policies is evaluated at each of the three V1 actuator gain values $\kappa \in \{0.65, 0.80, 1.00\}$ over 50 seeds with κ held fixed per episode. Crashed policies (runtime exception, subprocess error, or timeout) are scored at `ERROR_PENALTY_PER_SCENARIO` $= -200$ to match the GRPO training signal (same constant, imported from `src/runner.py`). Reference curves (blind PD, oracle) computed over the same 50 seeds.

Results

Table 3: Arm A: frozen one-shot Qwen vs. reference policies. Mean reward averaged over 20 generated policies × 50 seeds; max reward is the best mean reward achieved by any single generated policy. Oracle scales PD gains by $1/\kappa$; blind uses nominal gains regardless of κ.

κ	Qwen 1-shot (mean)	Qwen 1-shot (max)	Blind PD	Oracle PD
0.65	−112.0	−12.1	−9.6	111.2
0.80	−111.9	−12.1	45.9	111.4
1.00	−111.9	−12.1	111.5	111.5

Interpretation

- **Zero success rate across all κ.** Not a single generated policy achieves the simultaneous tolerance conditions over 50 seeds at any gain value, including $\kappa = 1.00$ where the blind PD achieves 100%.
- **Two failure modes.** Approximately 8/20 policies crash at runtime (hallucinated state keys such as `state["config"]`; scored at −200). The remaining 12/20 run but produce wrong control (reversed-sign bang-bang, pendulum dynamics used as a control law); their mean reward ≈ −12 to −16.
- **Best single policy: −12.1 at all κ.** Even the best generated policy scores well below blind PD at $\kappa = 0.80$ (+45.9) and $\kappa = 1.00$ (+111.5).
- **Gap to oracle: ≈ 223 reward points** (mean Qwen −112 vs. oracle +111). This is the training signal available to GRPO and is large enough to provide a clear learning gradient.
- **κ-invariant Qwen reward.** Flat mean and max across all three κ values confirms the model has no awareness of the hidden gain — it produces the same broken policies regardless of κ.

Reproduce

```
bash scripts/gcp_find_gpu_and_eval.sh arm_a_001 --wait \
```

```
-- --turns 1 --n-samples 20 --n-seeds 50
```

Results: `~/Downloads/lora4vr_eval_arm_a_001/arm_a_001/`. Crash penalty fix: PR #27.

2026-05-10 Arm B: frozen two-turn Qwen2.5-Coder-3B baseline

642400b · *Branch* `main`*; scripts* `scripts/eval_frozen_baseline.py`*,* `scripts/gcp_find_gpu_and_eval.sh`*. Run:* `arm_b_007`*, 20 samples per* κ*, 50 seeds, L4 GPU.*

Setup

Base Qwen2.5-Coder-3B-Instruct (no LoRA), two-turn protocol (`total_turns=2`), temperature 0.8. Turn 1 may be ```` ```policy ```` or ```` ```main ```` (arbitrary scratchpad code; a pre-instantiated `env` object calibrated to the evaluation κ is available). Turn 2 must be ```` ```policy ````. The model was run 20 times per $\kappa \in \{0.65, 0.80, 1.00\}$ (60 total samples). Each valid turn-2 policy is evaluated over 50 seeds with κ held fixed. Reference curves (blind PD, oracle) computed over the same 50 seeds as Arm A.

Results

Table 4: Arm B: frozen two-turn Qwen vs. reference policies. Success rate = fraction of 50 episodes where all four settling conditions are satisfied simultaneously. Blind PD uses nominal gains; Oracle PD scales gains by $1/\kappa$.

κ	Qwen 2-turn (success)	Blind PD (success)	Oracle PD (success)
0.65	0%	0%	100%
0.80	0%	46%	100%
1.00	0%	100%	100%

Turn-1 mode distribution (60 samples):

Mode	Count	Note
`invalid` (unrecognized fence)	45 (75%)	model emitted ```` ```python ```` tag
`policy`	12 (20%)	skipped the scratchpad entirely
none (empty response)	3 (5%)	

Failure breakdown (all 60 samples score at error floor or worse):

Failure mode	Count
Syntax error in turn-2 output (double-fence / garbled)	32
Runtime crash: `state['config']` hallucination	17
Runtime crash: other	1
Ran but bad control (below blind PD)	7
Empty code extracted from response	3

Interpretation

- **Zero success rate across all κ.** The model fails identically at $\kappa = 1.00$ where both blind PD (100%) and oracle PD (100%) succeed trivially. The failure is entirely protocol-driven, not task-difficulty-driven.
- **The model never uses `main` mode.** 100% of turn-1 outputs used a ```` ```python ```` fence tag (parsed as `invalid`) or produced an empty response. The calibration env was available but went completely unused because the frozen model has no prior for the two-turn protocol.

- **When turn 1 skips to policy (20%), all outputs crash.** Every one of the 12 samples that went straight to a turn-1 policy also hallucinated `state['config']` or `state['hidden']`. These keys do not exist in the state dict; the model is applying API assumptions from standard RL frameworks.
- **Turn-2 syntax errors dominate (32/60).** When turn 1 is `invalid`, the tool returns a rejection message. The model then consistently outputs another ‘‘‘`python`-tagged fence, which the runner cannot parse as a policy, causing a syntax error at validation.
- **Oracle PD confirms the task is solvable.** 100% success at all three κ values. The gap between Qwen frozen (0%) and oracle (100%) is entirely a *protocol learning* gap: the model must learn (a) to use ‘‘‘`main` to probe κ on turn 1, and (b) to reference only the state keys that actually exist.
- **Blind PD κ-sensitivity validates the probe necessity.** Blind PD drops from 100% at $\kappa = 1.00$ to 46% at $\kappa = 0.80$ and 0% at $\kappa = 0.65$, confirming that κ identification is genuinely required for reliable performance across the gain distribution.

Reproduce

```
bash scripts/gcp_find_gpu_and_eval.sh arm_b_007 --wait \
  -- --turns 2 --n-samples 20 --n-seeds 50
```

Results: `~/Downloads/lora4vr_eval_arm_b_007/arm_b_007/`. Debug-logging and per-sample `t1_mode`/`tool_output`/`t2_raw` fields: PR #35.

2026-05-10 Arm B revisited: parsing was a mask; the real gap is control reasoning, not protocol

`8943cbd` · *arm_b_008/009/010 across PRs #36, #37, #38, #39*

Setup

The previous Arm B entry (run `arm_b_007`) attributed the 0% success rate at every κ to a “protocol learning gap.” Inspection of the per-sample artefacts told a different story: 75% of turn-1 outputs were emitting ‘‘‘`python`-tagged fences (the model’s training-data prior), parsed as `mode=invalid` by a runner expecting tagged ‘‘‘`policy`/‘‘‘`main` fences. The 0% success rate was an artefact of the parser, not a measurement of two-turn reasoning. Three follow-up PRs progressively peeled the masking layers; each Arm B re-run revealed the next one.

PR #36 switched fence dispatch to ‘‘‘`python` + a `# MODE: policy|main` directive on the first non-blank line, made both seed envs return `(state, info)` from `reset()` (matching Gymnasium’s `step()` 5-tuple), and updated the runner templates. PR #37 added a minimal in-context example showing `state, _ = env.reset(); print(state)` in `# MODE: main` followed by a no-op `# MODE: policy` — deliberately uninformative to avoid the run-5 collapse fixed point. PR #38 regenerated the task JSONLs (whose embedded `env_code` was a stale snapshot of the env source), which PR #36 had not done.

Results

Interpretation

The 007 “protocol learning” diagnosis was wrong — or, more precisely, incomplete. Three independent failure modes had stacked, and only the outermost (parsing) was visible until the layer was peeled.

Layer 1 — 008. With the fence protocol fixed, all 60 samples parsed as `mode=policy` but every sample still hit the −200 floor. The new failure was state-key hallucination: every sample

Run	invalid	main	policy	above floor	best reward	success
arm_b_007 (pre-fix)	75%	0%	20%	0 / 60	−200	0%
arm_b_008 (PR #36)	0%	0%	100%	0 / 60	−200	0%
arm_b_009 (PR #37)	1.7%	27%	71%	0 / 60	−200	0%
arm_b_010 (PR #38, #39)	0%	27%	73%	30 / 60	≈ -10	0%

Table 5: Arm B turn-1 mode distribution and reward stats across four re-runs of the same prompt template under progressive bug fixes. "Above floor" counts samples whose mean reward exceeds the −200 per-scenario error penalty.

read nonexistent keys (`state['config']`, `state['hidden']`, `state['a_cmd']`, `EnvConfig()`) drawn from RL-codebase priors over the env source rather than from the `## Interface` section. Without an example, the model had no incentive to use `# MODE: main`: 0/60 probed.

Layer 2 — 009. The in-context example moved 16/60 (27%) samples into `mode=main`, but every probe crashed with `ValueError: too many values to unpack (expected 2, got 7)`. PR #36 had updated `slosh_tank_env.py` on disk to return `(state, info)` but had not regenerated `slosh_train.jsonl`, whose `env_code` field is a verbatim snapshot of the env source. The model wrote `state, _ = env.reset()` as taught by the example; the subprocess used the old single-return env code; the unpack failed; tool output was `[empty output]` plus a truncated `[main error]`. The model never saw the real state dict, so turn 2 fell back to hallucinated keys. Still 0% above floor.

Layer 3 — 010. With the JSONLs regenerated, probes succeed and return real state dicts. 30/60 samples now run to completion above the −200 floor; the best mean reward is ≈ -10 to -12 across all κ. Success rate stays at 0%, but for an entirely new reason: the policies that run are bad controllers, not crashed ones.

The capacity ceiling of the frozen 3B coder is now visible. The single best policy at $\kappa = 0.80$ reads the four-condition success criterion ($|x - g| < 0.12$, $|v| < 0.12$, $|\theta| < 0.07$, $|\omega| < 0.18$) and writes:

```
action = 0.1 * (abs(x-goal) + abs(v) + abs(theta) + abs(omega))
action = clip(action, -1, 1)
```

The output is always non-negative, so the cart only pushes one direction regardless of which side of the goal it starts on. The model read "minimise $|e|$" as "output is the magnitude of e," missing the connection to signed error in a control law. This is a reasoning gap, not a protocol gap.

The probe is similarly shape-only. Of the 16 main-mode samples in 010, only 3 escape the floor. Every probe I sampled calls `env.reset()` and prints `state.keys()` — the literal shape of the example. None probe κ (the actual hidden parameter) by stepping the env with a known acceleration and observing the resulting velocity change. The deliberately uninformative example taught the format but not what probing is for, which was the predicted tradeoff at PR #37.

Hypotheses / next steps

1. **Frozen-baseline sweep at larger / newer Qwen.** Run Arm A and Arm B on `Qwen2.5-Coder-7B-Instruct` (same family scale-up) and `Qwen3-4B-Instruct-2507` (newer training, similar size class). Both fit a slight L4 upgrade. The aim is to check whether the abs-error control bug and the shape-only probe persist at higher capacity, or whether they are 3B-specific.

2. **First slosh GRPO run.** The gap signal is now clean (Qwen-frozen ≈ -10 best vs. oracle $+111$) and large. Use the same config that worked for run 8 (LR $= 10^{-4}$, DAPO, $\beta = 0$), with the V1 actuator-gain unknown.
3. **Make the example slightly more informative.** Replace `print(state.keys())` with one canonical step (`state, _ = env.step(1.0)`) so the model sees what `step` returns and what a unit acceleration does to velocity. Risk: that one step becomes a GRPO attractor.
4. **Add a CI guard against env/JSONL drift.** The `env_code` embedding in `slosh_train.jsonl` silently diverged from `slosh_tank_env.py`. A pre-commit or CI hook that re-runs `build_*_tasks.py` and fails on diff would have caught the 009 regression at PR #36.

Open questions

- Will GRPO discover signed-error control without an explicit demonstration? Run 8 found a velocity-tracking template on the puck task without a prompt example; the slosh task is harder because four conditions must settle simultaneously.
- How sensitive is the result to the example's exact wording? One sample in 009 (sample 7) literally copied the strings "`[Tool output from turn 1]`" and "`Turn 2 --- final # MODE: policy:`" into its own response, suggesting the template-style framing of the example is read as a transcript to fill in rather than a pattern. The 010 logs do not show the same issue at scale, but the framing remains a candidate for simplification.
- Does the abs-error control bug reproduce on `Qwen3-4B` or does it lift? If it lifts, the slosh study needs to consider the model size as a controlled variable.

Reproduce

At `8943cbd`:

```
bash scripts/gcp_find_gpu_and_eval.sh arm_b_010 --wait \
  -- --turns 2 --n-samples 20 --n-seeds 50
```

Results: `~/Downloads/lora4vr_eval_arm_b_010/arm_b_010/`. Earlier intermediate runs at `arm_b_008/` and `arm_b_009/`. Relevant PRs: #36 (fence protocol + Gymnasium reset), #37 (in-context example), #38 (JSONL regen), #39 (auto-confirm VM teardown).

2026-05-10 Switching base model to Qwen3-4B-Instruct-2507; Arm A confirms the swap is worth it

`fd15c1f` · *arm_a_qwen3_4b_001, frozen baseline, 20 samples, 50 seeds*

Setup

The previous-entry analysis identified two failure modes in the frozen `Qwen2.5-Coder-3B-Instruct` baseline that were not load-bearing to the research question: (i) state-key hallucination (`state['config']`, `state['hidden']`, `EnvConfig()`) drawn from RL-codebase priors over the env source, and (ii) shape-only probing in Arm B. For paper legitimacy a base model that does not pre-trip on these specific artefacts was needed; the `Qwen` family's next major version, `Qwen3-4B-Instruct-2507` (August 2025 refresh), is the closest dense same-class candidate (4B vs 3B; no 3B exists in Qwen3 dense). PR #41 widened `load_model_for_generation` to accept HF repo ids so the swap became a single `--model-path` flag with no setup-script edits. Run `arm_a_qwen3_4b_001` re-runs the Arm A protocol from the earlier baseline entry on the new model.

Metric	**Qwen2.5-Coder-3B** (`arm_a_001`)	**Qwen3-4B-Instruct-2507** (`arm_a_qwen3_4b_001`)
Valid samples (no crash)	12 / 20	**20 / 20**
State-key hallucination crashes	$\approx 8/20$	**0**
Mean reward, $\kappa = 0.65$	-112	-76 (median)
Mean reward, $\kappa = 0.80$	-112	-87
Mean reward, $\kappa = 1.00$	-112	-86
Best single-sample reward (any κ)	-12.1	-11.7
Success rate, $\kappa = 0.65$	0%	0%
Success rate, $\kappa = 0.80$	0%	**0.5%**
Success rate, $\kappa = 1.00$	0%	**0.7%**

Table 6: Frozen Arm A: same prompt, same eval scenarios (20 generated policies $\times$ 50 seeds at each of three κ values).

Results

Interpretation

Three things shifted, with two clearly load-bearing for the study.

State-key hallucination is gone. Every one of 20 sampled policies reads only the documented state keys (t, x, v, θ, ω, *goal*, *steps_left*). The `state['config']/EnvConfig()` pattern that produced $\approx 40\%$ of the Qwen2.5 Arm A failures and dominated the previous Arm B re-runs does not reproduce on the new model.

First non-zero successes appear in single-turn. 1/20 samples at $\kappa = 1.00$ and 1/20 at $\kappa = 0.80$ produced occasional goal-and-settle episodes (1 success per 50 seeds = 0.7% / 0.5% respectively). The previous baseline had 0% across all κ, reflecting a hard wall that the new model crosses without any prompt or training change.

Mean reward improves by 25–36 *points.* Median per-sample mean reward shifts from -112 to roughly -86. The gain is broad (every sample now runs to completion above the -200 floor) rather than concentrated in a few stronger samples; the best single-sample reward changes only from -12.1 to -11.7.

The decision is to retire the Qwen2.5 family from the study and proceed on `Qwen3-4B-Instruct-2507` as the base model going forward. The 4B vs 3B size delta is the only visible confound; it is small enough to be acceptable and is acknowledged here for the paper write-up.

Hypotheses / next steps

1. **Run Arm B (`arm_b_qwen3_4b_001`).** The same protocol on two turns. Open question: does the model use `# MODE: main` more readily than the previous baseline (which sat at 27%), and does the probe content move beyond `print(state.keys())`?
2. **First slosh GRPO run on the new base.** The frozen baseline's success rate ($\leq 0.7\%$) leaves a clean and large gap to the oracle (100%) for GRPO to amplify, with the previous state-key-hallucination distraction removed.
3. **Update the eval pipeline default** from `models/hf_base/iter_000` to either pre-cache `Qwen3-4B-Instruct-2507` or document the new `--model-path` flag in onboarding.

Open questions

- Does the abs-error control bug (sum of absolute errors as the output, identified in the previous entry) reproduce on the new base? The mean-reward improvement is consistent with the bug persisting but being partly compensated by other factors; the new samples need direct inspection.
- How much of the gain is attributable to the parameter-count increase ($3 \to 4$ B, +33%) vs. the family/training change? Within scope to flag for paper reviewers, but a Qwen2.5-Coder-7B control run is not currently planned.

Reproduce

At `fd15c1f`:

```
bash scripts/gcp_find_gpu_and_eval.sh arm_a_qwen3_4b_001 --wait \
  -- --turns 1 --n-samples 20 --n-seeds 50 \
     --model-path Qwen/Qwen3-4B-Instruct-2507
```

Results: `~/Downloads/lora4vr_eval_arm_a_qwen3_4b_001/`. HF-repo-id support: PR #41.

2026-05-10 Frozen baseline summary on Qwen3-4B-Instruct-2507: naive, oracle, 1-turn, 2-turn

`cd1bc4e` · *arm_a_qwen3_4b_001 + arm_b_qwen3_4b_001, 20 samples per κ, 50 seeds*

Setup

With `Qwen3-4B-Instruct-2507` adopted as the study's base model, this entry consolidates the four frozen reference points the paper will cite directly: a naive blind PD controller, the oracle PD controller (gains scaled by $1/\kappa$), the model in a single-turn prompt (`arm_a_qwen3_4b_001`), and the model in the two-turn probe-then-adapt prompt (`arm_b_qwen3_4b_001`). The two-turn run completed shortly before this entry. All four were evaluated on the same 50 seeds and the same three actuator-gain values $\kappa \in \{0.65,\ 0.80,\ 1.00\}$ used in the previous Arm A/B work.

Results

κ	Naive PD	Oracle PD	Qwen3-4B 1-turn	Qwen3-4B 2-turn
0.65	0%	100%	0%	**0.7%** †
0.80	46%	100%	0.5%	0%
1.00	100%	100%	0.7%	0%

Table 7: Success rate (fraction of 50 episodes where all four settling conditions are satisfied simultaneously). Naive uses nominal PD gains; oracle scales gains by $1/\kappa$. Qwen rows are mean over 20 generated policies × 50 seeds. †Aggregate is 0.7% but driven by a single sample whose per-sample success rate was 14% — see Interpretation.

Turn-1 mode usage in Arm B. 60/60 samples (100%) opened with `# MODE: main`. The Qwen2.5-Coder-3B baseline at the same prompt was 27%.

Interpretation

Universal probing. Every Arm B sample probed in turn 1. Inspection of the raw outputs shows the probes are content-bearing, not just shape: most call `env.reset()` *and* step the env with a known acceleration (e.g. `env.step(0.0)`, `env.step(1.0)`) before printing observations, with

κ	1-turn min	1-turn med	1-turn max	2-turn med
0.65	−89.8	−76.3	−12.5	−57.7
0.80	−89.0	−86.8	−12.1	−74.2
1.00	−88.8	−86.5	−11.7	−68.2

Table 8: Per-sample mean-reward distribution (across 50 seeds), for the new model under both protocols. Best 2-turn sample at $\kappa = 0.65$ reached +5.7 — the first positive frozen-baseline reward in any slosh run on any model.

comments like "probing the environment to gain insights about the hidden actuator gain." The Qwen2.5 baseline never produced this behaviour without further intervention.

Two-turn modestly outperforms one-turn on mean reward, but not on success rate. Median reward improves at every κ (−57.7 vs. −76.3 at 0.65; −74.2 vs. −86.8 at 0.80; −68.2 vs. −86.5 at 1.00). Success-rate differences across protocols are within sampling noise at $n = 20$: the 1-turn 0.5%/0.7% successes at $\kappa = 0.80, 1.00$ disappeared in 2-turn while a 0.7% success appeared at $\kappa = 0.65$.

First positive reward. The best Arm B sample at $\kappa = 0.65$ achieved a mean reward of +5.7 over 50 seeds, with 14% per-seed success rate — the first time any frozen baseline policy on the slosh task crossed zero reward. The same sample is the only single policy in the run that hits the four-condition criterion repeatably.

Gap to oracle remains large. Aggregate success rate is $\leq 0.7\%$ across all κ versus oracle's 100%, leaving ≥ 99.3 percentage points of headroom and roughly 175–195 reward points of mean shortfall. The frozen-baseline ceiling is now characterised cleanly: probing is universal and informative, control reasoning is the residual blocker, and the gap is exactly the kind of signal GRPO is designed to amplify.

Hypotheses / next steps

1. **First slosh GRPO run on Qwen3-4B-Instruct-2507.** Same config as run 8 (LR $= 10^{-4}$, DAPO, $\beta = 0$), same V1 unknown (κ only). The four-row table above is the reference the training run will be plotted against.
2. **Inspect the $\kappa = 0.65$, +5.7 outlier sample.** What did its probe reveal, and did the turn-2 controller use that information? If yes, this is the first concrete instance of the probe-then-adapt mechanism in the wild.
3. **Document outlier-driven aggregate metric.** The $\kappa = 0.65$ Arm B aggregate (0.7%) is one sample with 14% success, not 20 samples averaging 0.7%. The paper write-up should report both the aggregate and the per-sample distribution so this isn't read as broad capability.

Open questions

- Why does $\kappa = 0.65$ (the hardest condition) get the only Arm B success while $\kappa = 0.80, 1.00$ regress relative to 1-turn? The 1-turn / 2-turn variance at $n = 20$ is large; a larger seed budget would clarify whether this is a real protocol effect or sampling noise.
- How much of the universal probing is the in-context example being copied vs. a genuine model-side preference? The example shows `env.reset(); print(state)`; the observed probes extend that with `env.step()` and explanatory comments, which suggests at least partial generalisation.

Reproduce

At `cd1bc4e`:

```
bash scripts/gcp_find_gpu_and_eval.sh arm_a_qwen3_4b_001 --wait \
  -- --turns 1 --n-samples 20 --n-seeds 50 \
     --model-path Qwen/Qwen3-4B-Instruct-2507

bash scripts/gcp_find_gpu_and_eval.sh arm_b_qwen3_4b_001 --wait \
  -- --turns 2 --n-samples 20 --n-seeds 50 \
     --model-path Qwen/Qwen3-4B-Instruct-2507
```

Results: `~/Downloads/lora4vr_eval_arm_{a,b}_qwen3_4b_001/`.

2026-05-10 Arm B turn-1 probe analysis: the gate opens but the value is not extracted

`cd1290a` · *static analysis of all 60 turn-1* `main` *bodies in arm_b_qwen3_4b_001*

Setup

The aggregate Arm B numbers (100% `# MODE: main` usage, 0.7% success at $\kappa = 0.65$ driven by a single +5.7-reward sample) raised two questions the per-sample table did not answer: *what* is the model probing in turn 1, and *does the probe content feed the turn-2 policy*? The 60 turn-1 `t1_raw` bodies were grepped for calls to `env.reset()` / `env.step(...)`, presence of `for`/`while` loops, the actions passed to `step`, and any code that names or computes the actuator gain. The killer sample was inspected by hand to see whether its turn-2 policy referenced turn-1 output.

Results

Probe shape distribution. All 60 samples open with `state, _ = env.reset()` followed by a `print` of the initial state. Beyond that:

Pattern	Count	Description
D — reset-only	39 / 60	literal example body, no further probing
A — reset + stepped trajectory	20 / 60	at least one `env.step(a)`, often in a loop
B — reset + unrolled step calls	1 / 60	multiple hardcoded `step` calls, no loop
C — reset + loop without step	0 / 60	—

Probe content within the active 21. Of the 21 samples that call `env.step`, none try a sweep of distinct accelerations to back out the actuator gain. Action values are dominated by the hardcoded literals 0.5, −0.5, 1.0. Only a single sample tests $a = 0.0$ (the natural-dynamics baseline that would isolate the gravity / coupling effects from the actuator path). 60/60 samples include a comment stating they are probing "to gain information about the hidden actuator gain κ"; 1/60 contains code that references `actuator_gain` / `kappa` / `k_est` or computes a $\Delta v/(a \cdot \Delta t)$ ratio. The intent is stated in the comment, the arithmetic never happens.

The killer sample did not probe. The Arm B winner at $\kappa = 0.65$ (mean reward +5.7, 14% per-seed success rate) is a Pattern-D probe — `env.reset()` followed by `print(state)` and nothing else. Its turn-2 policy is a four-state PD with signed errors: action $= 0.8\,(g-x)/0.12 - 0.2\,v/0.12 - 0.3\,\theta/0.07 - 0.1\,\omega/0.18$, clipped. The abs-error bug from the previous-model 2-turn baseline is gone, and the controller addresses all four success conditions explicitly. The most active probe in the run (sample 0, full stepped trajectory under both positive and negative actions, scored −57.7) did not use its probe data in turn 2 either; it was a weaker controller of the same family.

Interpretation

The probe-then-adapt protocol is *nominal but not operative*. Every sample passes the format gate (`# MODE: main` on turn 1, `# MODE: policy` on turn 2, valid state-key reads), and the prompt's instruction to use the pre-instantiated `env` for probing is followed by a wide majority either literally (Pattern D) or loosely (Pattern A). But the value the protocol is supposed to deliver — a κ estimate that conditions the turn-2 controller — never materialises. Two independent observations support this:

1. No sample computes a κ estimate from probe data, even when the data needed (a known commanded action, an observed velocity change, the dt visible in the env source) is on screen.
2. The single sample that did achieve repeatable success on the hardest κ did so without a probe at all. Its win came from cleaner control reasoning, not from cross-turn information transfer.

The Arm B headline "0.7% success rate at $\kappa = 0.65$" is therefore correctly read as a control-reasoning result that happens to occur in a two-turn rollout, not as an instance of probe-then-adapt. The mean-reward improvement of 2-turn over 1-turn ($\approx$ 12–18 points per κ) is consistent with the model writing slightly better controllers in the second turn after seeing the actual state shape printed in the first — a weak, format-side effect, not an information-gathering one.

Hypotheses / next steps

1. **The gap GRPO needs to close is now sharp.** A 2-turn rollout that actually estimates κ in turn 1 and adapts in turn 2 should beat reset-only Pattern D on every κ, not just one outlier. GRPO with $G_2 \geq 2$ branched continuations gives the trainer the signal to reward probe content that improves turn-2 reward.
2. **Probe-aware reward bonus (optional).** Augment the reward with a small term proportional to the mutual information between the printed turn-1 stdout and the resulting turn-2 reward, or a simpler proxy (e.g. "did turn 2 read the printed numbers"). Risk: gameable. Worth keeping in the back pocket if vanilla GRPO plateaus on Pattern D.
3. **Make the example marginally more informative.** Replace the example's `print("state keys:", sorted(state.keys()))` line with a single `state, _ = env.step(1.0)` followed by a print, so the demonstrated probe shows *what* a step does. Trades a smaller chance of GRPO collapsing onto the example for a larger chance of the model generalising probe content. Decision deferred until after the first GRPO run.

Open questions

- Does the model already *understand* that $\Delta v/a$ at $\theta \approx 0$, $\omega \approx 0$ on a single step gives κ, but choose not to use it? Or is the missing arithmetic genuine? A targeted prompt asking the same model "estimate κ from one step" (outside the slosh study) would answer this cleanly.
- How much of the 100% probe rate is generalisation vs. literal example copying? 65% Pattern D (reset-only, like the example) and 33% Pattern A (extends the example with `step`) suggest the answer is roughly two-thirds copy, one-third generalisation.

Reproduce

At `cd1290a`: the analysis is a static grep over the `t1_raw` field of each entry in `arm_b_summary.json`, plus manual inspection of the highest `mean_reward` sample at each κ. No additional code; the patterns used were `env\.reset\(` and `env\.step\(` for shape, and `kappa|actuator_gain|k_est` for κ-naming. Source: `~/Downloads/lora4vr_eval_arm_b_qwen3_4b_001/arm_b_qwen3_4b_001/arm_b_summary.js`

2026-05-15 L4 OOM at production sizing despite paper's B=16 memory budget; forward-side temperature scaling allocates a transient fp32 logits duplicate

288b686 · *run* ***dedup-prod-l4****, fix in PR #67*

Setup

After landing the branched-trainer dedup fix (PR #63) the per-step backward batch dropped from B=256 (TRL `RepeatSampler` expansion times branched fan-out) back to the paper-intended $B{=}G_1G_2{=}16$. A smoke run at B=4 confirmed the pipeline worked end-to-end on the 24 GB L4, peaking around 60% of GPU memory. The next step was the full 23-iteration production run with `configs/loop_grpo_branched_slosh.yaml` ($G_1{=}G_2{=}4$, `max_turn_tokens`=400, *pdtbs*=16).

Results

The first run (`dedup-prod-l4`, no `loss_micro_batch_size` set) completed the step-0 rollout in 114 s, finished the turn-1 `compute_loss` pass, then OOMed inside the turn-2 pass at TRL's `logits = logits / self.temperature` (`trl/trainer/grpo_trainer.py:1022`) with *CUDA out of memory. Tried to allocate 3.23 GiB. Total 22.03 GiB, free 1.77 GiB, in use 20.26 GiB.* Re-launching the same config with `branched.loss_micro_batch_size: 4` cleared the OOM; the re-launched run (`dedup-prod-l4-mb4`) reached step 5 with $\widehat{V}_0$ progressing from −88.5 at step 0 to −36.0 at step 5 over ∼27 minutes wall-clock.

	Production (B=16, T=400)		Smoke (pre-dedup)
	no microbatching	mb = 4	B_{eff}=16, T=100
LM-head logits tensor (fp32)	3.89 GiB	0.97 GiB	0.97 GiB
Peak at `logits/temperature` op	∼7.8 GiB	∼2 GiB	∼2 GiB
Outcome on 24 GiB L4	OOM step 0	runs	runs (∼60% mem)

Interpretation

The paper's Appendix A memory accounting (Table 2) was correct as far as it went: the per-turn LM-head *gradient* tensor at B=16, T=400 is ≈ 3.9 GiB, and the two-turn loss decomposition keeps that under *max* rather than *sum* across turns. What it failed to account for is that the *forward* also peaks at the same size, and TRL's `logits = logits / self.temperature` is not in place — a same-shape duplicate is allocated before the original tensor is freed. The logits are also held in fp32 for softmax numerical stability rather than bf16, which doubles the byte budget over a naive estimate. The net effect at the divide step is two 3.89 GiB tensors simultaneously live, ∼7.8 GiB in flight for that single op. Stacked on ∼8 GiB of model weights, residual KV cache from the turn-2 generation phase that PyTorch had not yet released, and the turn-1 backward's residual activation cache, the L4's 22 GiB usable budget did not have room for the second 3.23 GiB allocation.

The micro-batching path (PR #62) exists for exactly this case. With `loss_micro_batch_size: 4`, each per-turn pass forwards 4-row chunks instead of the full B=16, so the logits tensor shrinks linearly to ≈ 0.97 GiB per chunk and the divide-step peak is bounded by ∼2 GiB. The gradient identity holds (`num_items_in_batch` is the global model-token count, passed unchanged to every chunk, so the chunk-summed loss is bit-equivalent to the single-pass loss). The empirical cost is ∼5% wall-clock from the extra LM-head kernel launches; rollout wall-clock at production sizing is dominated by generation (∼120 s/step) so the chunked backward is invisible at the step level.

This run is the first empirical validation of the per-turn micro-batching feature. Until now its memory savings were predicted from Appendix A arithmetic; the OOM provides the counterfactual

at the exact configuration the feature was designed for.

Hypotheses / next steps

1. **Update Appendix A.** The current table accounts for the backward LM-head gradient but not the forward fp32 logits tensor or the transient duplicate from `logits/self.temperature`. Either add a row that includes those, or replace the table with a worked example at production sizing showing why B=16, T=400 does not fit on a 24 GiB L4 without per-row chunking.
2. **Default `loss_micro_batch_size: 4` for L4 configs.** Done for the slosh prod config in PR #67. The other branched configs (`loop_grpo_branched.yaml`, `smoke_test_grpo_branched.yaml`) should follow.
3. **Measure cost on A100.** On a 40 GB+ card the divide step fits without chunking; quantify the chunked-vs-unchunked wall-clock there so the recommendation is hardware-aware.

Open questions

- Is there a way to convince TRL (or upstream HF) to do `logits.div_(temperature)` in place when entropy computation does not require the unscaled values? That would eliminate the transient duplicate without any user-side change.
- What is the next bottleneck on L4 once the LM-head forward is chunked? Generation-phase KV cache for turn-2 (G_2=4 samples, prompt+turn1+tool $\approx$ 1500 tokens) is the next-largest sustained allocation; if a tighter T budget or fewer turn-2 samples is later required, that is the term to attack.

Reproduce

At `288b686`: launch the production config on a fresh L4 with `bash scripts/gcp_find_gpu_and_train.sh configs/loop_grpo_branched_slosh.yaml dedup-prod-l4-mb4 --accelerator l4`. To reproduce the OOM, comment out the `loss_micro_batch_size: 4` line in `configs/loop_grpo_branched_slosh.yam` and re-launch — the crash occurs in step 0's turn-2 `compute_loss` pass within $\sim$3 minutes.